DS – HOMEWORK 4

1. Implement a Stack using Queue

```
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = 0, rear = -1;

void enqueue(int x)
{
   queue[++rear] = x;
}

int dequeue()
{
   return queue[front++];
}

void push(int x)
{
   int i, size = rear - front + 1;

   enqueue(x);

   for (i = 0; i < size; i++)
```

```
        enqueue(dequeue());
}

int pop()
{
    if (front > rear)
    {
        printf("Stack Underflow\n");
        return -1;
    }

    return dequeue();
}

int peek()
{
    if (front > rear)
    {
        printf("Stack is empty\n");
        return -1;
    }

    return queue[front];
}

void display()
{
```

```
    int i;

    if (front > rear)
    {
        printf("Stack is empty\n");
        return;
    }

    printf("Stack: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);

    printf("\n");
}

int main()
{
    int n, i, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &value);
```

```
        push(value);
    }

    display();

    printf("Popped element: %d\n", pop());
    printf("Top element: %d\n", peek());

    display();

    return 0;
}
```

Program Output:

```
C:\Users\Student\Documents\stack_using_queue\stack_using_queue.exe
Enter number of elements: 5
Enter elements:
10 20 30 40 50
Stack: 50 40 30 20 10
Popped element: 50
Top element: 40
Stack: 40 30 20 10
```

2. Implement a Queue using Stack

```
#include <stdio.h>

#define MAX 100
```

```
int stack1[MAX], stack2[MAX];
int top1 = -1, top2 = -1;

void push1(int x)
{
   stack1[++top1] = x;
}

void push2(int x)
{
   stack2[++top2] = x;
}

int pop1()
{
   return stack1[top1--];
}

int pop2()
{
   return stack2[top2--];
}

void enqueue(int x)
{
   push1(x);
}
```

```
int dequeue()
{
   int x;

   if (top1 == -1 && top2 == -1)
   {
      printf("Queue Underflow\n");
      return -1;
   }

   if (top2 == -1)
   {
      while (top1 != -1)
      {
         x = pop1();
         push2(x);
      }
   }

   return pop2();
}

void display()
{
   int i;
```

```
    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue: ");

    for (i = top2; i >= 0; i--)
        printf("%d ", stack2[i]);

    for (i = 0; i <= top1; i++)
        printf("%d ", stack1[i]);

    printf("\n");
}

int main()
{
    int n, i, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for (i = 0; i < n; i++)
```

```
    {
        scanf("%d", &value);
        enqueue(value);
    }

    display();

    printf("Deleted element: %d\n", dequeue());

    display();

    return 0;
}
```

```
C:\Users\Student\Documents\queue_using_stack\queue_using_stack.exe
Enter number of elements: 5
Enter elements:
10 20 30 40 50
Queue: 10 20 30 40 50
Deleted element: 10
Queue: 20 30 40 50
```

3. Browser History using Stack

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char history[MAX][100];
```

```
int top = -1;

void visit(char page[])
{
    if (top == MAX - 1)
    {
        printf("History is full\n");
        return;
    }

    strcpy(history[++top], page);
}

void back()
{
    if (top <= 0)
    {
        printf("No previous page\n");
        return;
    }

    top--;
    printf("Current Page: %s\n", history[top]);
}

void currentPage()
{
```

```
    if (top == -1)
        printf("No page visited\n");
    else
        printf("Current Page: %s\n", history[top]);
}

void display()
{
    int i;

    printf("Browser History:\n");

    for (i = top; i >= 0; i--)
        printf("%s\n", history[i]);
}

int main()
{
    visit("google.com");
    visit("youtube.com");
    visit("github.com");
    visit("stackoverflow.com");

    currentPage();

    printf("\nGoing Back...\n");
    back();
```

```
    printf("\nGoing Back...\n");
    back();

    printf("\n");
    display();

    return 0;
}
```

```
C:\Users\Student\Documents\browser_history\browser_history.exe
Current Page: stackoverflow.com

Going Back...
Current Page: github.com

Going Back...
Current Page: youtube.com

Browser History:
youtube.com
google.com
```

4. Write a C program to sort a stack using temporary stack

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int temp[MAX];
int top = -1;
int ttop = -1;
```

```
void push(int x)
{
   stack[++top] = x;
}

int pop()
{
   return stack[top--];
}

void sortStack()
{
   int x;

   while (top != -1)
   {
      x = pop();

      while (ttop != -1 && temp[ttop] > x)
      {
         stack[++top] = temp[ttop--];
      }

      temp[++ttop] = x;
   }

   while (ttop != -1)
```

```
        stack[++top] = temp[ttop--];
}

void display()
{
    int i;

    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);

    printf("\n");
}

int main()
{
    push(30);
    push(10);
    push(50);
    push(20);
    push(40);

    printf("Original Stack: ");
    display();

    sortStack();

    printf("Sorted Stack: ");
```

```
    display();

    return 0;
}
```

```
C:\Users\Student\Documents\sort_stack\sort_stack.exe
Original Stack: 40 20 50 10 30
Sorted Stack: 50 40 30 20 10
```

5. Stock Span Problem

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

int main()
{
    int price[MAX], span[MAX];
    int n, i;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter stock prices:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &price[i]);
```

```
    for (i = 0; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;

        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }

    printf("Stock Span:\n");

    for (i = 0; i < n; i++)
        printf("%d ", span[i]);

    printf("\n");

    return 0;
}
```



6. Nearest Element to Right with Higher Frequency

```
 #include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

int main()
{
    int arr[MAX], freq[MAX], result[MAX];
    int n, i, j;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
/* Calculate frequency of each element */
for (i = 0; i < n; i++)
{
    freq[i] = 0;

    for (j = 0; j < n; j++)
    {
        if (arr[i] == arr[j])
            freq[i]++;
    }
}

/*
   Stack contains indices.
   Find the nearest element to the right
   having greater frequency.
*/

top = -1;

for (i = n - 1; i >= 0; i--)
{
    while (top != -1 && freq[stack[top]] <= freq[i])
        top--;

    if (top == -1)
```

```
            result[i] = -1;
        else
            result[i] = arr[stack[top]];

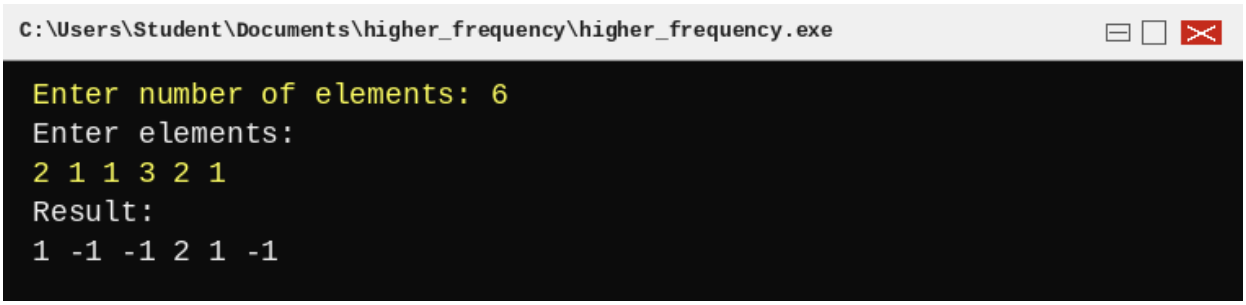
        stack[++top] = i;
    }

    printf("Result:\n");

    for (i = 0; i < n; i++)
        printf("%d ", result[i]);

    printf("\n");

    return 0;
}
```



7. Given a pattern containing only I's and D's, print the minimum number using Stack

```
#include <stdio.h>

#define MAX 100
```

```
int stack[MAX];
int top = -1;

int main()
{
    char pattern[MAX];
    int i, j;
    int n;

    printf("Enter I/D pattern: ");
    scanf("%s", pattern);

    n = 0;

    while (pattern[n] != '\0')
        n++;

    printf("Minimum number: ");

    for (i = 0; i <= n; i++)
    {
        stack[++top] = i + 1;

        if (i == n || pattern[i] == 'I')
        {
            while (top != -1)
```

```
            printf("%d", stack[top--]);
        }
    }

    printf("\n");

    return 0;
}
```



8. Given an array of digits (0 to 9), form two numbers whose sum is minimum

```
#include <stdio.h>

void sort(int arr[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (arr[i] > arr[j])
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

int main()
{
    int arr[100];
    int n, i;
    int num1 = 0, num2 = 0;
    int sum;
```

```
    printf("Enter number of digits: ");
    scanf("%d", &n);

    printf("Enter digits:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

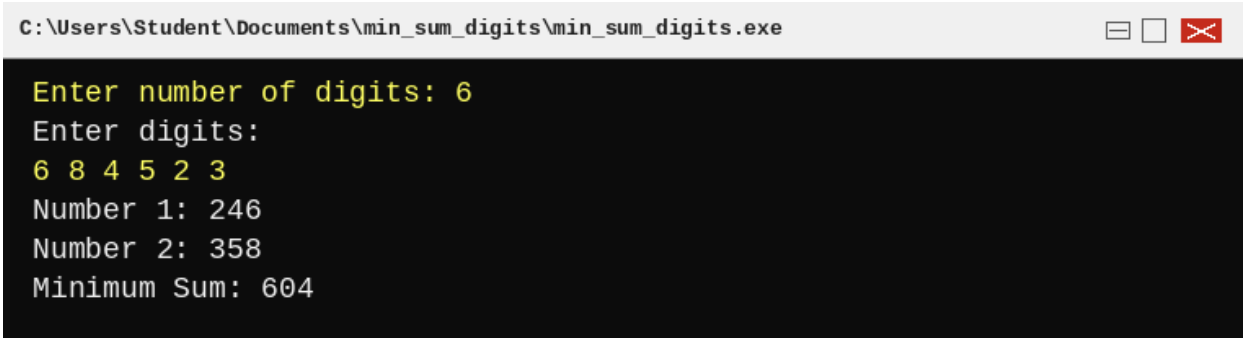
    sort(arr, n);

    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + arr[i];
        else
            num2 = num2 * 10 + arr[i];
    }

    sum = num1 + num2;

    printf("Number 1: %d\n", num1);
    printf("Number 2: %d\n", num2);
    printf("Minimum Sum: %d\n", sum);

    return 0;
}
```


C:\Users\Student\Documents\min_sum_digits\min_sum_digits.exe
Enter number of digits: 6
Enter digits:
6 8 4 5 2 3
Number 1: 246
Number 2: 358
Minimum Sum: 604

9. Find the first negative integer for every contiguous window of size K

```
#include <stdio.h>

#define MAX 100

int main()
{
    int arr[MAX];
    int queue[MAX];
    int front = 0, rear = -1;
    int n, k, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");

    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter window size: ");
    scanf("%d", &k);

    for (i = 0; i < n; i++)
    {
        /* Store index of negative elements */
        if (arr[i] < 0)
```

```
        queue[++rear] = i;

    /* Remove elements outside current window */
    while (front <= rear && queue[front] <= i - k)
        front++;

    /* Print answer when window is complete */
    if (i >= k - 1)
    {
        if (front <= rear)
            printf("%d ", arr[queue[front]]);
        else
            printf("0 ");
    }
  }

  printf("\n");

  return 0;
}
```

```
C:\Users\Student\Documents\negative_in_window\negative_in_window.exe
Enter number of elements: 5
Enter array elements:
-8 2 3 -6 10
Enter window size: 2
-8 0 -6 -6
```

10. Given a matrix of oranges, find the minimum time required for all oranges to become rotten

```
#include <stdio.h>

#define MAX 100

typedef struct
{
   int row;
   int col;
} Node;

Node queue[MAX * MAX];

int front = 0;
int rear = -1;

int main()
{
   int grid[MAX][MAX];
   int rows, cols;
   int i, j;
   int fresh = 0;
   int minutes = 0;

   int dr[] = {-1, 1, 0, 0};
   int dc[] = {0, 0, -1, 1};
```

```
printf("Enter number of rows and columns: ");
scanf("%d %d", &rows, &cols);

printf("Enter matrix:\n");

for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        scanf("%d", &grid[i][j]);

        if (grid[i][j] == 1)
            fresh++;

        if (grid[i][j] == 2)
        {
            queue[++rear].row = i;
            queue[rear].col = j;
        }
    }
}

while (front <= rear && fresh > 0)
{
    int currentSize = rear - front + 1;
```

```
    for (i = 0; i < currentSize; i++)
    {
        int r = queue[front].row;
        int c = queue[front].col;

        front++;

        for (j = 0; j < 4; j++)
        {
            int nr = r + dr[j];
            int nc = c + dc[j];

            if (nr >= 0 && nr < rows &&
                nc >= 0 && nc < cols &&
                grid[nr][nc] == 1)
            {
                grid[nr][nc] = 2;
                fresh--;

                queue[++rear].row = nr;
                queue[rear].col = nc;
            }
        }
    }

    minutes++;
}
```

```
    if (fresh == 0)
        printf("Minimum time required: %d\n", minutes);
    else
        printf("-1\n");

    return 0;
}
```

